

Real-Time Fraud Detection ML Scoring Layer

An offline-trained XGBoost classifier scores a live Kafka transaction stream in real time through Spark Structured Streaming — turning a descriptive monitoring pipeline (Project 1) into a predictive one.

Repository · github.com/DiegoTDDD/realtime-fraud-detection-ml

Builds on · github.com/DiegoTDDD/realtime-fraud-streaming (Project 1)

Diego Leonardo Alves de Andrade

Executive summary

Project 1 built the streaming backbone for card-transaction fraud detection — event flow, windowed aggregation, a descriptive dashboard — and closed with an explicit roadmap: turn the descriptive monitor into a predictive one. This project does that. I trained an XGBoost classifier offline on a purpose-generated dataset, integrated it into Spark Structured Streaming as an independent second consumer of Project 1's Kafka topic, and shipped a live dashboard that reports the model's real-world precision and recall — not just transaction counts.

0.9925 PR-AUC (offline test set)	96.6% / 95.8% Recall / Precision @ threshold	150,000 Training transactions generated	3 Feature-engineering iterations tested	5 Infra failures diagnosed & fixed	
XGBoost	scikit-learn	Spark Structured Streaming	Redpanda (Kafka API)	Parquet	Streamlit + Plotly

The problem I set out to solve

Project 1's producer injects three classic fraud patterns — an abnormally high charge, a burst of small “probe” charges on one card, and the same card transacting in distant cities minutes apart — each carrying a ground-truth label for exactly this purpose. The question was whether a model trained offline on that signal could serve the live stream in real time, at a class balance where accuracy is meaningless, and whether I could prove — not just claim — that it wasn't only catching the easiest, highest-volume pattern.

Guiding question

Can an XGBoost classifier trained offline on synthetically injected fraud patterns generalize to a live Spark Structured Streaming job, hold up under a ~2–7% fraud rate without inflating precision by ignoring the rare patterns, and do so with training-serving feature parity strict enough that the offline metrics are trustworthy?

Why the training data had to be generated, not sampled

Project 1's real Bronze history had only 189 fraud examples — too few for a stable precision/recall read, especially split three ways by pattern. Running the live producer for hours to accumulate more was the obvious option, but it conflates two different jobs: simulating a live feed and building a historical training set. I reused the producer's exact transaction-generation logic (same card population, same three fraud patterns, same base rates) in a standalone batch script that writes 150,000 transactions directly to Parquet in seconds, with no Kafka or Spark involved.

Decision I'd defend

I didn't touch Project 1's producer code to get more data faster. Modifying a shared dependency to serve one downstream need is the kind of shortcut that quietly breaks the other consumer. Reusing its logic in an isolated script kept both projects independently correct.

How I designed the system

```
Producer (Project 1) --> Redpanda (Kafka API) --> Spark Structured Streaming
card txns + labels      topic: transactions
                        foreachBatch:
                          1. feature engineering
                          2. XGBoost scoring
                          3. write predictions
                          |
                          v
                        Parquet (predictions)
                          |
                          v
                        Streamlit dashboard
                        (live precision/recall)

training/model.joblib loaded ONCE, before the stream starts
(not reloaded per micro-batch)
```

The scoring job is an **independent second consumer** of Project 1's topic — it doesn't modify or depend on Project 1's own Bronze/Gold pipeline running at the same time, only on the topic existing. That separation is what let me build and iterate on this project without ever touching Project 1's code.

Class imbalance: weights, not resampling

I used **class weights** (`scale_pos_weight` in XGBoost), not undersampling or SMOTE. Undersampling throws away real signal; oversampling risks leakage in a time-ordered dataset. Class weights preserve the real distribution and are the standard choice for fraud detection in production. I evaluated with **PR-AUC**, since accuracy is meaningless at a 2–7% fraud rate — a model that never predicts fraud already scores 93–98%.

The features themselves are deliberately narrow: `amount`, `distance_km_from_home`, `tx_count_last_5min`, `time_since_last_tx`, `hour_of_day`, and one-hot category. I left merchant out on purpose — it's a high-cardinality, effectively random string with no real predictive signal here, and feeding it to the model would risk overfitting to noise rather than learning the actual fraud patterns.

A chronological split, not a random one

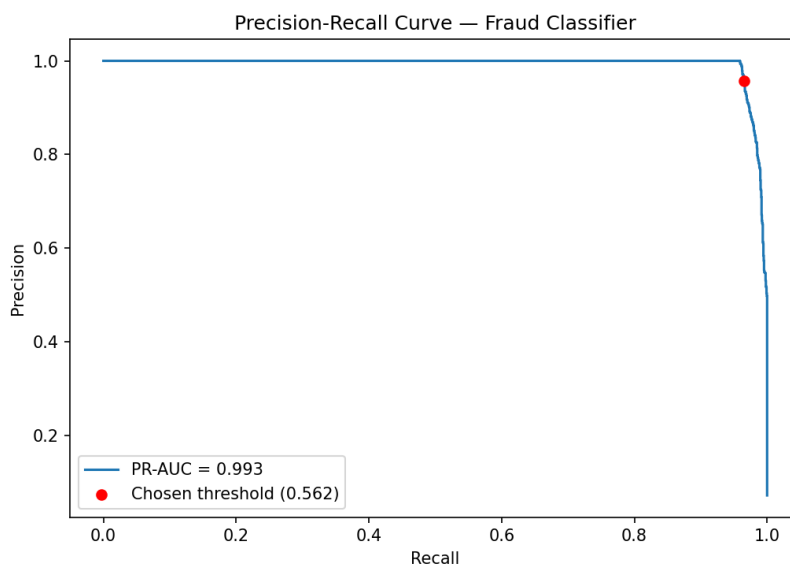
Two engineered features (`tx_count_last_5min`, `time_since_last_tx`) depend on nearby transactions in time. A random 80/20 split would leak information across the boundary — a test-set row's velocity feature could be computed from training-set neighbors. I split the earliest 80% of transactions for training and the latest 20% for testing, which is also the honest mirror of how the model is actually used: trained on the past, scoring the future.

Model development & evaluation

The default 0.5 threshold, and even the aggregate-F1-optimal threshold, both get dominated by whichever fraud pattern has the most examples — `card_testing`, at roughly 83% of fraud volume in this dataset. I instead capped false positives with a 95% precision floor (an alert budget a real fraud team could act on) and, within that budget, chose the threshold that maximizes the *average* recall across the three fraud types equally — so the rarer patterns can't be silently ignored just because they're outnumbered.

Train: 120,000 rows (9,114 fraud) · Test: 30,000 rows (2,147 fraud), chronologically split. `scale_pos_weight` computed at 12.17 from the training split's actual class ratio, not a guessed value.

Metric	Value
PR-AUC (held-out, chronological test set)	0.9925
Precision @ threshold (0.5616)	95.75%
Recall @ threshold	96.55%
Recall — <code>high_amount</code>	96.2%
Recall — <code>card_testing</code>	100%
Recall — <code>impossible_travel</code>	64.7%



Precision-recall curve on the held-out test set. The chosen threshold (red dot) sits just past the curve's near-vertical drop — recall beyond this point costs precision fast.

Feature engineering as an iterative, falsifiable process

`high_amount` and `card_testing` were straightforward to separate. `impossible_travel` was not, and getting there — including two rejected approaches — is worth documenting because the process is more informative than the final number alone.

Approach tried	Recall (<code>impossible_travel</code>)	Why it stalled
Binary <code>is_unusual_city</code> flag	0.59–0.65	Too coarse — can't tell a nearby trip from a cross-country jump.

Approach tried	Recall (impossible_travel)	Why it stalled
implied_speed_kmh (distance from previous tx / elapsed time)	0.62	This dataset's per-card cadence means the previous transaction is often hours away, so elapsed-time-aware speed doesn't reliably flag a geographically anomalous but time-isolated transaction.
distance_km_from_home (Haversine, inferred home city) — final	0.65	Best of the three tested; kept.

The decision I'm most upfront about

0.65 recall on impossible_travel is a real gap, not a rounding error, and I chose not to keep chasing it past a third iteration. Root cause: only ~190 test examples of this pattern (high sampling variance), and 15% of Project 1's legitimate transactions also occur away from the card's home city, so distance alone has an irreducible false-negative rate without more data or a stronger signal — e.g. an actual flight-time feasibility model. I'd rather state that clearly than let a reviewer find it first.

Real-time serving design

Feature engineering at serving time has to exactly mirror training, or the model sees a different feature distribution live than it was trained on — **training-serving skew**, one of the most common and most expensive bugs in production ML. Two features needed particular care because they depend on data outside a single micro-batch:

```
def score_batch(batch_df, batch_id):
    pdf = batch_df.toPandas()
    for _, row in pdf.iterrows():
        # distance_km_from_home: reuses the exact home-city mapping
        # computed once at training time (card_home_city.json)
        home = HOME_CITY.get(row.card_id)
        distance = haversine_km(row.city, home)

        # tx_count_last_5min / time_since_last_tx: need each card's
        # recent history, which can span multiple micro-batches.
        # Kept in a process-local dict, updated here.
        history = CARD_HISTORY.setdefault(row.card_id, deque())
        ...

    X = pdf[FEATURE_COLS]
    pdf['fraud_probability'] = MODEL.predict_proba(X)[: , 1]
```

The velocity-feature state lives in a Python dict local to the driver process, not a distributed state store. That's a documented simplification, not an oversight: it only works because this job runs as a single local process, matching how every project in this portfolio runs. A production deployment across multiple workers would need a proper state store — Spark's own stateful streaming operators, or Redis — and I say so directly rather than let the code imply it scales further than it does.

One operational detail worth flagging: the job reads with `startingOffsets: earliest`, so on a fresh start it reprocesses the topic's entire retained history in one large first batch before settling into small, continuous ones — expected Kafka semantics, not a bug, but worth knowing before watching the first batch take longer than the rest.

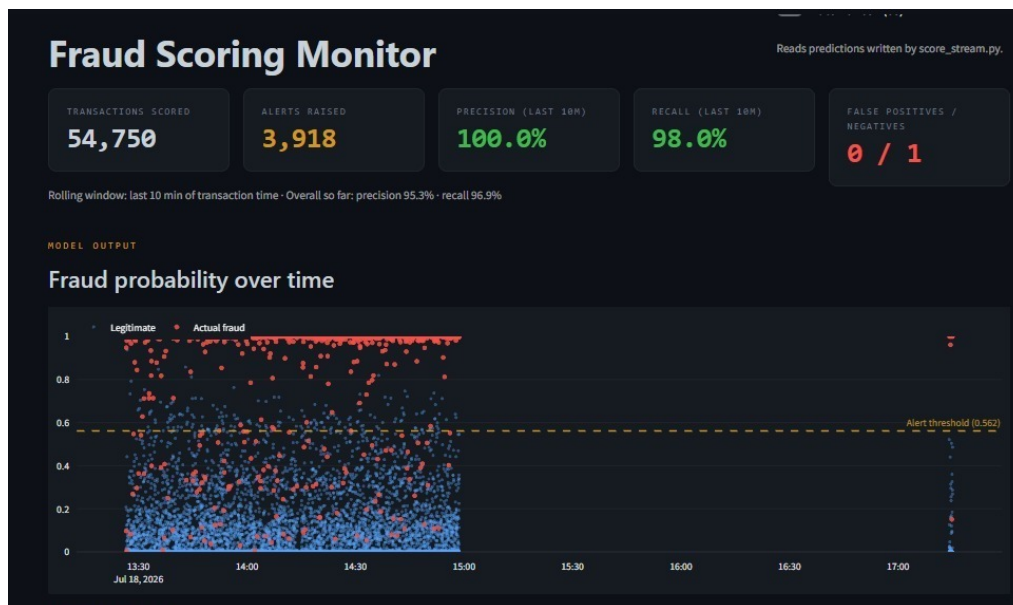
Getting it running: five infrastructure failures

None of these were model bugs — they were the real friction of running Spark Structured Streaming with a Python foreachBatch sink on Windows, against a broker that had been restarted. I'm including them because diagnosing infrastructure failures from raw stack traces, not just writing the happy-path code, is the part of this work that doesn't show up in a metrics table.

Failure	Root cause	Fix
SparkContext failed to initialize	Hadoop native binaries (winutils.exe, hadoop.dll) missing on Windows; a naive download link served an HTML page instead of the binary.	Downloaded from the correct raw.githubusercontent.com URL; verified with file(1) before retrying.
Redpanda topic didn't exist	The broker container has no persistent volume, so a host reboot wiped it. The scoring job's Kafka source fails outright on a missing topic, not gracefully.	Re-ran the producer first — it recreates the topic on its first publish — then started scoring.
"Python worker failed to connect back"	Spark's foreachBatch spawns a Python worker via the system PATH; on Windows this resolved to a Microsoft Store App Execution Alias stub, not the real conda interpreter.	Set PYSPARK_PYTHON / PYSPARK_DRIVER_PYTHON explicitly to sys.executable before creating the SparkSession.
CANNOT_DETERMINE_TYPE on write	Spark infers a pandas-to-Spark schema per micro-batch; a micro-batch with zero fraud makes fraud_type entirely null, which has no inferable type.	Declared an explicit output StructType, removing per-batch schema inference.
gh / winutils commands not found after install	Windows PATH updates from an installer don't propagate to an already-open shell.	Located the binary directly and exported it onto PATH for the session.

Live dashboard results

The dashboard reads what the scoring job writes and reports model quality, not just transaction volume — possible here specifically because Project 1's producer injects ground-truth labels into every simulated transaction. Most real-world fraud dashboards can't show live accuracy, because they don't have an oracle to check predictions against.



54,750 live-scored transactions: rolling 10-minute precision/recall (100% / 98%) alongside the running overall figures (95.3% / 96.9%), both consistent with the offline test-set numbers on the previous page. Fraud probability scatter shows a clean separation around the alert threshold (0.562).



Live recall by fraud type alongside the offline training numbers. All three patterns appear on both bars — the model isn't only catching card_testing live, matching the offline evaluation.

ALERTS

Flagged transactions

Time	Card	Amount \$	Category	City	Score	Actual type	Correct
2026-07-18 13:44:51.955766	card_0001	\$10,375.82	groceries	Sao Paulo	1.000	high_amount	✓
2026-07-18 14:28:25.316824	card_0015	\$3.96	entertainment	Curitiba	1.000	card_testing	✓
2026-07-18 14:23:19.854496	card_0106	\$2.07	entertainment	Manaus	1.000	card_testing	✓
2026-07-18 14:22:45.659399	card_0044	\$2.85	entertainment	Fortaleza	1.000	card_testing	✓
2026-07-18 13:46:07.217798	card_0065	\$3.78	entertainment	Salvador	1.000	card_testing	✓
2026-07-18 13:58:53.752281	card_0056	\$0.86	entertainment	Sao Paulo	1.000	card_testing	✓
2026-07-18 13:36:26.357211	card_0143	\$1.03	entertainment	Rio de Janeiro	1.000	card_testing	✓
2026-07-18 13:37:17.735313	card_0192	\$4.47	entertainment	Salvador	1.000	card_testing	✓
2026-07-18 14:20:11.464433	card_0132	\$1.85	entertainment	Salvador	1.000	card_testing	✓
2026-07-18 13:38:12.780555	card_0028	\$3.58	entertainment	Belo Horizonte	1.000	card_testing	✓

54,750 transactions scored · pipeline: producer → Redpanda → Spark Structured Streaming (foreachBatch) → XGBoost → Parquet → this dashboard

The flagged-transaction feed, ranked by model confidence, with a correct/incorrect indicator against ground truth — a live audit trail a fraud analyst could actually use.

What this project demonstrates

Applied ML on imbalanced data — Class weights over resampling, PR-AUC as the primary metric, a threshold strategy chosen for the minority pattern instead of the aggregate score.

A falsifiable feature-engineering process — Three approaches tested and measured against the same held-out set, with the two rejected ones kept in the write-up alongside the reasoning.

Streaming ML serving — A model integrated into Spark Structured Streaming via `foreachBatch`, with training-serving feature parity enforced deliberately, not assumed.

Honest documentation of a real constraint — Process-local state for velocity features, stated as a single-process limitation rather than glossed over.

Real infrastructure debugging — Five distinct failures diagnosed from stack traces and fixed with root-cause explanations, on a genuinely uncooperative Windows/Spark setup.

Honest limitations

impossible_travel recall (64.7%) is the clearest open gap, with root cause documented on page 5 — closing it further needs more labeled examples of that pattern or a genuinely physical feasibility feature (e.g. real flight-time constraints), not another threshold tweak. **Process-local state** for velocity features means this scoring job, as built, runs correctly only as a single process; a multi-worker deployment needs a shared state store. **Training data is synthetic**, generated by reusing Project 1's simulation logic in batch rather than sampled from a real card network — appropriate for a portfolio demonstration, not a claim of production-scale validation.

Technical reference

Stage	Component	Purpose
Model	XGBoost, scikit-learn	Gradient-boosted classifier, class-weighted

Stage	Component	Purpose
Training data	Batch generation script	Reuses Project 1's fraud logic; 150k rows in seconds
Serving	Spark Structured Streaming (PySpark 3.5)	foreachBatch scoring against the live topic
Broker	Redpanda (Kafka API)	Reused as-is from Project 1
Storage	Parquet	Scored predictions
Presentation	Streamlit + Plotly	Live precision/recall dashboard
Runtime	Python 3.11 (conda), Windows + Git Bash	Local, no cloud dependency

Data: synthetic transactions generated locally by Project 1's producer logic, with fraud labels injected deliberately. Repository: github.com/DiegoTDDD/realtime-fraud-detection-ml · Builds on: github.com/DiegoTDDD/realtime-fraud-streaming